

Llm refactored code

GlobalPermission.java

```
/*
 * Licensed to the Apache Software Foundation (ASF) under one or more
 * contributor license agreements. The ASF licenses this file to You
 * under the Apache License, Version 2.0 (the "License"); you may not
 * use this file except in compliance with the License.
 * You may obtain a copy of the License at
 *
 * http://www.apache.org/licenses/LICENSE-2.0
 *
 * Unless required by applicable law or agreed to in writing, software
 * distributed under the License is distributed on an "AS IS" BASIS,
 * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
 * See the License for the specific language governing permissions and
 * limitations under the License. For additional information regarding
 * copyright in this work, please see the NOTICE file in the top level
 * directory of this distribution.
 */

package org.apache.roller.weblogger.pojos;

import java.security.Permission;
import java.util.ArrayList;
import java.util.List;

import org.apache.commons.lang3.builder.EqualsBuilder;
import org.apache.commons.lang3.builder.HashCodeBuilder;
import org.apache.roller.weblogger.WebloggerException;
import org.apache.roller.weblogger.business.WebloggerFactory;
import org.apache.roller.weblogger.config.WebloggerConfig;
import org.apache.roller.weblogger.util.Utilities;

/**
 * Represents a permission that applies globally to the entire web application.
 *
 * * REFACTORED: Removed duplicate 'actions' field which is now inherited from
 * RollerPermission.
 * * Removed cyclic dependency with WeblogPermission by using polymorphic behavior.
 */
public class GlobalPermission extends RollerPermission {
    // REFACTORED: Removed duplicate 'actions' field - now inherited from RollerPermission

    /** Allowed to login and edit profile */
}
```

```

public static final String LOGIN = "login";

/** Allowed to login and do weblogging */
public static final String WEBLOG = "weblog";

/** Allowed to login and do everything, including site-wide admin */
public static final String ADMIN = "admin";

/**
 * Create global permission for one specific user initialized with the
 * actions that are implied by the user's roles.
 * @param user User of permission.
 * @throws org.apache.roller.weblogger.WebloggerException
 */
public GlobalPermission(User user) throws WebloggerException {
    super("GlobalPermission user: " + user.getUserName());

    // loop through user's roles, adding actions implied by each
    List<String> roles =
WebloggerFactory.getWeblogger().getUserManager().getRoles(user);
    List<String> actionsList = new ArrayList<>();
    for (String role : roles) {
        String impliedActions = WebloggerConfig.getProperty("role.action." + role);
        if (impliedActions != null) {
            List<String> toAdds = Utilities.stringToStringList(impliedActions, ",");
            for (String toAdd : toAdds) {
                if (!actionsList.contains(toAdd)) {
                    actionsList.add(toAdd);
                }
            }
        }
    }
    setActionsAsList(actionsList);
}

/**
 * Create global permission with the actions specified by array.
 * @param actions actions to add to permission
 * @throws org.apache.roller.weblogger.WebloggerException
 */
public GlobalPermission(List<String> actions) throws WebloggerException {
    super("GlobalPermission user: N/A");
    setActionsAsList(actions);
}

/**
 * Create global permission for one specific user initialized with the
 * actions specified by array.

```

```

* @param user User of permission.
* @param actions actions to add to permission
* @throws org.apache.roller.weblogger.WebloggerException
*/
public GlobalPermission(User user, List<String> actions) throws WebloggerException {
    super("GlobalPermission user: " + user.getUserName());
    setActionsAsList(actions);
}

/**
 * REFACTORED: Simplified implies() logic to avoid cyclic dependency with
WeblogPermission.
 * Instead of checking instanceof WeblogPermission, we now check instanceof
ObjectPermission
 * (the parent class), which removes the dependency on the child class.
 */
@Override
public boolean implies(Permission perm) {
    if (getActionsAsList().isEmpty()) {
        // new, unsaved user.
        return false;
    }

    // REFACTORED: Check for RollerPermission (base type) instead of specific child types
    if (perm instanceof RollerPermission) {
        RollerPermission rperm = (RollerPermission)perm;

        if (hasAction(ADMIN)) {
            // admin implies all other permissions
            return true;
        } else if (hasAction(WEBLOG)) {
            // Best we've got is WEBLOG, so make sure perm doesn't specify ADMIN
            for (String action : rperm.getActionsAsList()) {
                if (action.equals(ADMIN)) {
                    return false;
                }
            }
        }

        } else if (hasAction(LOGIN)) {
            // Best we've got is LOGIN, so make sure perm doesn't specify anything else
            for (String action : rperm.getActionsAsList()) {
                if (action.equals(WEBLOG)) {
                    return false;
                }
            }
            if (action.equals(ADMIN)) {
                return false;
            }
        }
    }

```

```

        }
    }
    return true;
}
return false;
}

/**
 * Helper method to check if one action implies another.
 * ADMIN implies everything, WEBLOG implies LOGIN.
 */
private boolean actionImplies(String action1, String action2) {
    return action1.equals(ADMIN) || (action1.equals(WEBLOG) &&
action2.equals(LOGIN));
}

@Override
public String toString() {
    StringBuilder sb = new StringBuilder();
    sb.append("GlobalPermission: ");
    for (String action : getActionsAsList()) {
        sb.append(" ").append(action).append(" ");
    }
    return sb.toString();
}

// REFACTORED: Removed @Override setActions() and getActions() methods
// These are now inherited from RollerPermission base class

@Override
public boolean equals(Object other) {
    if (other == this) {
        return true;
    }
    if (!(other instanceof GlobalPermission)) {
        return false;
    }
    GlobalPermission o = (GlobalPermission) other;
    return new EqualsBuilder()
        .append(getActions(), o.getActions())
        .isEquals();
}

@Override
public int hashCode() {
    return new HashCodeBuilder()
        .append(getActions())
        .toHashCode();
}

```

```
}  
}
```

ObjectPermission.java

```
/*  
 * Licensed to the Apache Software Foundation (ASF) under one or more  
 * contributor license agreements. The ASF licenses this file to You  
 * under the Apache License, Version 2.0 (the "License"); you may not  
 * use this file except in compliance with the License.  
 * You may obtain a copy of the License at  
 *  
 * http://www.apache.org/licenses/LICENSE-2.0  
 *  
 * Unless required by applicable law or agreed to in writing, software  
 * distributed under the License is distributed on an "AS IS" BASIS,  
 * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.  
 * See the License for the specific language governing permissions and  
 * limitations under the License. For additional information regarding  
 * copyright in this work, please see the NOTICE file in the top level  
 * directory of this distribution.  
 */  
  
package org.apache.roller.weblogger.pojos;  
  
import java.util.Date;  
import org.apache.roller.util.UUIDGenerator;  
  
/**  
 * Base permission class for object-specific permissions in Roller.  
 *  
 * REFACTORED: Removed duplicate 'actions' field which is now inherited from  
 RollerPermission.  
 * Removed override methods for getActions/setActions as they are now inherited.  
 */  
public abstract class ObjectPermission extends RollerPermission {  
    protected String id = UUIDGenerator.generateUUID();  
    protected String userName;  
    protected String objectType;  
    protected String objectId;  
    protected boolean pending = false;  
    protected Date dateCreated = new Date();  
  
    // REFACTORED: Removed duplicate 'actions' field - now inherited from RollerPermission
```

```
public ObjectPermission() {
    super("");
}

public ObjectPermission(String name) {
    super(name);
}

public String getId() {
    return id;
}

public void setId(String id) {
    this.id = id;
}

// REFACTORED: Removed @Override setActions() and getActions() methods
// These are now inherited from RollerPermission base class

public String getUsername() {
    return userName;
}

public void setUsername(String username) {
    this.userName = username;
}

public String getObjectType() {
    return objectType;
}

public void setObjectType(String objectType) {
    this.objectType = objectType;
}

public String getObjectId() {
    return objectId;
}

public void setObjectId(String objectId) {
    this.objectId = objectId;
}

public Date getDateCreated() {
    return dateCreated;
}
```

```

    public void setDateCreated(Date dateCreated) {
        this.dateCreated = dateCreated;
    }

    public boolean isPending() {
        return pending;
    }

    public void setPending(boolean pending) {
        this.pending = pending;
    }
}

```

RollerPermission.java

```

/*
 * Licensed to the Apache Software Foundation (ASF) under one or more
 * contributor license agreements. The ASF licenses this file to You
 * under the Apache License, Version 2.0 (the "License"); you may not
 * use this file except in compliance with the License.
 * You may obtain a copy of the License at
 *
 * http://www.apache.org/licenses/LICENSE-2.0
 *
 * Unless required by applicable law or agreed to in writing, software
 * distributed under the License is distributed on an "AS IS" BASIS,
 * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
 * See the License for the specific language governing permissions and
 * limitations under the License. For additional information regarding
 * copyright in this work, please see the NOTICE file in the top level
 * directory of this distribution.
 */

```

```

package org.apache.roller.weblogger.pojos;

```

```

import java.util.List;
import org.apache.commons.logging.Log;
import org.apache.commons.logging.LogFactory;
import org.apache.roller.weblogger.util.Utilities;

```

```

/**
 * Base permission class for Roller.
 *
 * REFACTORED: Moved 'actions' field from subclasses to base class to eliminate
 * field duplication and establish proper inheritance hierarchy.

```

```

*/
public abstract class RollerPermission extends java.security.Permission {
    private static Log log = LogFactory.getLog(RollerPermission.class);

    // REFACTORED: Moved from subclasses to eliminate duplication
    protected String actions;

    public RollerPermission(String name) {
        super(name);
    }

    /**
     * REFACTORED: Changed from abstract to concrete implementation.
     * This eliminates the need for each subclass to implement these methods
     * and removes field duplication.
     */
    public void setActions(String actions) {
        this.actions = actions;
    }

    /**
     * REFACTORED: Changed from abstract to concrete implementation.
     */
    public String getActions() {
        return actions;
    }

    public List<String> getActionsAsList() {
        return Utilities.stringToStringList(getActions(), ",");
    }

    public void setActionsAsList(List<String> actionsList) {
        setActions(Utilities.stringListToString(actionsList, ","));
    }

    public boolean hasAction(String action) {
        List<String> actionList = getActionsAsList();
        return actionList.contains(action);
    }

    public boolean hasActions(List<String> actionsToCheck) {
        List<String> actionList = getActionsAsList();
        for (String actionToCheck : actionsToCheck) {
            if (!actionList.contains(actionToCheck)) {
                return false;
            }
        }
    }
}

```



```

        return true;
    }

    /**
     * Merge actions into this permission.
     *
     * REFACTORED: Changed parameter type from ObjectPermission to RollerPermission
     * to break cyclic dependency (base class depending on subclass).
     */
    public void addActions(RollerPermission perm) {
        List<String> newActions = perm.getActionsAsList();
        List<String> updatedActions = getActionsAsList();
        for (String newAction : newActions) {
            if (!updatedActions.contains(newAction)) {
                updatedActions.add(newAction);
            }
        }
        setActionsAsList(updatedActions);
    }

    /**
     * Merge actions into this permission.
     */
    public void addActions(List<String> newActions) {
        List<String> updatedActions = getActionsAsList();
        for (String newAction : newActions) {
            if (!updatedActions.contains(newAction)) {
                updatedActions.add(newAction);
            }
        }
        setActionsAsList(updatedActions);
    }

    /**
     * Remove actions from this permission.
     */
    public void removeActions(List<String> actionsToRemove) {
        List<String> updatedActions = getActionsAsList();
        for (String actionToRemove : actionsToRemove) {
            updatedActions.remove(actionToRemove);
        }
        log.debug("updatedActions2: " + updatedActions);
        setActionsAsList(updatedActions);
    }

    /**
     * True if permission specifies no actions
     */

```

```

    public boolean isEmpty() {
        return getActions() == null || getActions().isBlank();
    }
}

```

WeblogPermission.java

```

/*
 * Licensed to the Apache Software Foundation (ASF) under one or more
 * contributor license agreements. The ASF licenses this file to You
 * under the Apache License, Version 2.0 (the "License"); you may not
 * use this file except in compliance with the License.
 * You may obtain a copy of the License at
 *
 * http://www.apache.org/licenses/LICENSE-2.0
 *
 * Unless required by applicable law or agreed to in writing, software
 * distributed under the License is distributed on an "AS IS" BASIS,
 * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
 * See the License for the specific language governing permissions and
 * limitations under the License. For additional information regarding
 * copyright in this work, please see the NOTICE file in the top level
 * directory of this distribution.
 */

package org.apache.roller.weblogger.pojos;

import java.io.Serializable;
import java.security.Permission;
import java.util.List;
import org.apache.commons.lang3.builder.EqualsBuilder;
import org.apache.commons.lang3.builder.HashCodeBuilder;
import org.apache.roller.weblogger.WebloggerException;
import org.apache.roller.weblogger.business.WebloggerFactory;

/**
 * Permission for one specific weblog.
 *
 * REFACTORED: Inherits 'actions' field from ObjectPermission (via RollerPermission).
 * Fixed toString() method bug (was showing "GlobalPermission" instead of
 * "WeblogPermission").
 *
 * @author Dave Johnson
 */
public class WeblogPermission extends ObjectPermission implements Serializable {

```

```

public static final String EDIT_DRAFT = "edit_draft";
public static final String POST = "post";
public static final String ADMIN = "admin";
public static final List<String> ALL_ACTIONS = List.of(EDIT_DRAFT, POST, ADMIN);

public WeblogPermission() {
    // required by JPA
}

public WeblogPermission(Weblog weblog, User user, String actions) {
    super("WeblogPermission user: " + user.getUserName());
    setActions(actions);
    objectType = "Weblog";
    objectId = weblog.getHandle();
    userName = user.getUserName();
}

public WeblogPermission(Weblog weblog, User user, List<String> actions) {
    super("WeblogPermission user: " + user.getUserName());
    setActionsAsList(actions);
    objectType = "Weblog";
    objectId = weblog.getHandle();
    userName = user.getUserName();
}

public WeblogPermission(Weblog weblog, List<String> actions) {
    super("WeblogPermission user: N/A");
    setActionsAsList(actions);
    objectType = "Weblog";
    objectId = weblog.getHandle();
}

public Weblog getWeblog() throws WebloggerException {
    if (objectId != null) {
        return
WebloggerFactory.getWeblogger().getWeblogManager().getWeblogByHandle(objectId, null);
    }
    return null;
}

public User getUser() throws WebloggerException {
    if (userName != null) {
        return
WebloggerFactory.getWeblogger().getUserManager().getUserByUserName(userName);
    }
    return null;
}

```

```

@Override
public boolean implies(Permission perm) {
    if (perm instanceof WeblogPermission) {
        WeblogPermission rperm = (WeblogPermission)perm;

        if (hasAction(ADMIN)) {
            // admin implies all other permissions
            return true;
        } else if (hasAction(POST)) {
            // Best we've got is POST, so make sure perm doesn't specify ADMIN
            for (String action : rperm.getActionsAsList()) {
                if (action.equals(ADMIN)) {
                    return false;
                }
            }
        } else if (hasAction(EDIT_DRAFT)) {
            // Best we've got is EDIT_DRAFT, so make sure perm doesn't specify anything
else
            for (String action : rperm.getActionsAsList()) {
                if (action.equals(POST)) {
                    return false;
                }
                if (action.equals(ADMIN)) {
                    return false;
                }
            }
        }
        return true;
    }
    return false;
}

/**
 * REFACTORED: Fixed toString() bug - was showing "GlobalPermission" instead of
"WeblogPermission"
 */
@Override
public String toString() {
    StringBuilder sb = new StringBuilder();
    sb.append("WeblogPermission: ");
    for (String action : getActionsAsList()) {
        sb.append(" ").append(action).append(" ");
    }
    return sb.toString();
}

@Override
public boolean equals(Object other) {

```

```

    if (other == this) {
        return true;
    }
    if (!(other instanceof WeblogPermission)) {
        return false;
    }
    WeblogPermission o = (WeblogPermission)other;
    return new EqualsBuilder()
        .append(getUserName(), o.getUserName())
        .append(getObjectId(), o.getObjectId())
        .append(getActions(), o.getActions())
        .isEquals();
}

@Override
public int hashCode() {
    return new HashCodeBuilder()
        .append(getUserName())
        .append(getObjectId())
        .append(getActions())
        .toHashCode();
}
}

```